

ZAVRŠNI MINI PROJEKAT

Docker + docker-compose + CI Pipeline

Edukativni projekat za QA timove



60 min predavanje



60-120 min samostalni rad

AGENDA

Šta ćemo proći

01

Cilj projekta

Šta ćemo naučiti i napraviti

02

Scenario i arhitektura

Realan QA primer sa 3 servisa

03

Struktura fajlova

Organizacija projekta

04

API aplikacija

Node.js + Express + PostgreSQL

05

Testovi

Automatizovani testovi sa Jest

06

Docker konfiguracija

Dockerfile za API i testove

07

Docker Compose

Orkestracija 3 servisa

08

Lokalno pokretanje

Testiranje pre CI-a

09

GitHub Actions Pipeline

Automatizacija u CI/CD

10

QA diskusija i zadatak

Mogući problemi i proširenja

Cilj Projekta



Šta treba da napravimo?

Aplikaciju koja demonstrira **kompletnu QA infrastrukturu** — od lokalnog razvoja do automatizovanog testiranja u CI pipeline-u.



Šta ćemo naučiti?

- ✓ Kako povezati više servisa u jednu aplikaciju
- ✓ Kako koristiti Docker za izolaciju servisa
- ✓ Kako automatizovati testove u CI pipeline-u



Komponente projekta

1

Backend servis

Node.js API sa Express framework-om

2

Baza podataka

PostgreSQL za perzistentno skladište

3

Testovi

Automatizovani testovi sa Jest

4

CI Pipeline

GitHub Actions za automatizaciju

SCENARIO

Realan QA Primer



Opis aplikacije

Imamo **jednostavan API za upravljanje korisnicima**. API omogućava osnovne CRUD operacije nad korisnicima.



Dodavanje korisnika

POST /users - kreira novog korisnika



Lista korisnika

GET /users - vraća sve korisnike



QA zadaci

QA tim želi da proveri:

1

Da li API pravilno odgovara na zahteve

2

Da li baza podataka ispravno čuva podatke

3

Da li pipeline automatski pokreće testove



Tehnologije koje koristimo



Docker

Kontejnerizacija servisa



docker-compose

Orkestracija više kontejnera



Test Automation

Jest + Axios za testove



GitHub Actions

CI/CD automatizacija



Ključna ideja: Sve komponente rade zajedno u izolovanim kontejnerima, što omogućava konzistentno testiranje na bilo kojem okruženju.

Arhitektura Projekta



SERVIS 1

API

 Node.js

 Express.js

 Port 3000



SERVIS 2

Database

 PostgreSQL 15

 Persistent storage

 Port 5432



SERVIS 3

Test Runner

 Jest

 Axios

 Automatsko pokretanje

Tok Podataka



TEST

Šalje HTTP zahteve



HTTP Request



API

Obraduje zahteve



SQL Query



DATABASE

Čuva podatke



Svi servisi su izolovani u Docker kontejnerima i komuniciraju preko definisanih mreža

Struktura Fajlova

project/

app/

server.js

package.json

tests/

api.test.js

package.json

Dockerfile

docker-compose.yml

Dockerfile

.env

.github/

workflows/

test.yml

app/

Sadrži **Node.js API aplikaciju**. server.js je glavni fajl sa Express serverom i PostgreSQL konekcijom.

tests/

Sadrži **automatske testove**. api.test.js koristi Jest i Axios za testiranje API-ja.

.github/workflows/

Sadrži **GitHub Actions workflow**. test.yml definiše CI pipeline koji se pokreće na push i pull request.

Napomena: Dockerfile se nalazi u root direktorijumu za API, a tests/Dockerfile za testove.

server.js - Node.js + Express + PostgreSQL

```
const express = require("express") const {Pool} = require("pg") const app = express()
app.use(express.json()) // PostgreSQL konekcija preko environment varijabli const pool =
new Pool({ host: process.env.DB_HOST, user: process.env.DB_USER, password:
process.env.DB_PASS, database: process.env.DB_NAME, port: 5432 }) // GET /users - vraća
sve korisnike app.get("/users", async (req, res) => { const result = await pool.query("select *
from users") res.json(result.rows) }) // POST /users - dodaje novog korisnika app.post("/users",
async (req, res) => { const {name} = req.body await pool.query("insert into users(name)
values($1)", [name] ) res.json({status: "ok"}) }) app.listen(3000, () => { console.log("server
running") })
```

 **Express.js** - minimalistički web framework za Node.js

package.json

```
{ "dependencies": { "express": "^4", "pg": "^8" } }
```

Ključne karakteristike

- 1 Environment varijable**
DB_HOST, DB_USER, DB_PASS, DB_NAME
- 2 Connection pooling**
Efikasno upravljanje konekcijama
- 3 Async/await**
Moderna obrada asinhronih operacija

 **PostgreSQL** - moćna open-source relacijska baza podataka

api.test.js - Jest + Axios

```
const axios = require("axios") // Test: Kreiranje korisnika i provera liste
test("create user", async () => { // Korak 1: Kreiraj korisnika
  await axios.post("http://app:3000/users", {name: "John"}) // Korak 2: Dohvati listu korisnika
  const response = await axios.get("http://app:3000/users") // Korak 3: Proveri da li postoji 1 korisnik
  expect(response.data.length).toBe(1) })
```

 tests/package.json

```
{ "dependencies": { "axios": "^1", "jest": "^29" }, "scripts": { "test": "jest" } }
```

Tok testa

- 1 Kreiraj korisnika**
POST /users sa imenom "John"
- 2 Dohvati listu**
GET /users za sve korisnike
- 3 Verifikuj**
Očekuj tačno 1 korisnika u listi

Zašto ovaj test?

- ✓ End-to-end test – proverava ceo tok
- ✓ Proverava API i bazu – integracioni test
- ✓ Jednostavan i brz – idealan za CI



http://app:3000 – Docker mreža omogućava komunikaciju preko imena servisa

Docker Konfiguracija



Dockerfile (API)

```
# Koristimo Node.js 18 slikuFROM node:18 # Postavi radni direktorijumWORKDIR /app #  
Kopiraj package.jsonCOPY app/package.json . # Instaliraj zavisnostiRUN npm install #  
Kopiraj ostale fajloveCOPY app . # Pokreni aplikacijuCMD ["node", "server.js"]
```



tests/Dockerfile

```
FROM node:18 WORKDIR /tests COPY tests/package.json . RUN npm install COPY tests .  
CMD ["npm", "test"]
```

i Objasnjenje komandi

FROM node:18

Bazna slika sa Node.js 18

WORKDIR /app

Postavlja radni direktorijum

COPY source dest

Kopira fajlove u kontejner

RUN npm install

Izvršava komandu prilikom build-a

CMD ["node", "server.js"]

Komanda koja se pokreće pri startu



Layer caching: Docker kešira svaki sloj za brže build-ove

Environment Variable - .env



.env fajl

```
# PostgreSQL konfiguracijaDB_HOST=db DB_USER=test DB_PASS=test  
DB_NAME=testdb
```

🛡️ Zašto .env?

- ✓ **Bezbednost**
Lozinke nisu u kodu
- ✓ **Fleksibilnost**
Različite vrednosti za različita okruženja
- ✓ **Jednostavnost**
Lako menjanje bez rebuild-a

🏠 Objašnjenje varijabli

DB_HOST=db

Ime servisa baze u Docker mreži

DB_USER=test

Korisničko ime za PostgreSQL

DB_PASS=test

Lozinka za PostgreSQL

DB_NAME=testdb

Ime baze podataka



VAŽNO: .env fajl ne treba commit-ovati u git! Dodajte ga u .gitignore.

Docker Compose - docker-compose.yml

```
version: "3.9"
services:
  # 1. Baza podataka
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: test
      POSTGRES_PASSWORD: test
      POSTGRES_DB: testdb
    volumes:
      - dbdata:/var/lib/postgresql/data
  # 2. API aplikacija
  app:
    build: .
    ports:
      - "3000:3000"
    env_file:
      - .env
    depends_on:
      - db
  # 3. Test runner
  test:
    build: ./tests
    depends_on:
      - app
    volumes:
      - dbdata:
```

Servis: db

image: Koristi PostgreSQL 15 sliku

environment: Postavlja kredencijale

volumes: Perzistentno skladište

Servis: app

build: Build-uje iz Dockerfile

ports: Mapira port 3000

env_file: Učitava .env

depends_on: Čeka bazu

Servis: test

build: Build-uje iz tests/Dockerfile

depends_on: Čeka API

 **depends_on** osigurava redosled pokretanja

Lokalno Pokretanje



Komanda za pokretanje

```
docker-compose up --build
```

☰ Šta se dešava?

1

Build-uje se API image

Iz Dockerfile u root direktorijumu

2

Build-uje se test image

Iz tests/Dockerfile

3

Pokreće se baza

PostgreSQL kontejner

4

Pokreće se API

Čeka bazu (depends_on)

5

Pokreću se testovi

Čeka API (depends_on)



Očekivani rezultat

```
Creating network "project_default"... Creating volume "project_dbdata"... Creating project_db_1 ... done
Creating project_app_1 ... done Creating project_test_1 ... done> jestPASS api.test.js ✓ create user
(125ms) Test Suites: 1 passed, 1 total Tests: 1 passed, 1 total
```



Korisne komande

docker-compose down

Zaustavlja i uklanja kontejnere

docker-compose logs

Prikazuje logove svih servisa

docker-compose ps

Prikazuje status kontejnera

GitHub Actions Pipeline



.github/workflows/test.yml

```
name: QA pipeline
on: push: pull_request:
jobs: test:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3
- name: start containers
run: docker-compose up --build --abort-on-container-exit
- name: stop containers
run: docker-compose down
```

🔑 Triggers (pokretači)



push

Kod se push-uje na bilo koji branch



pull_request

Kreira se pull request

☰ Šta pipeline radi?

1

Checkout repo

Preuzima kod iz repozitorijuma

2

Pokrene docker-compose

Build-uje i pokreće sve servise

3

Pokrene testove

Automatski pokreće test runner

4

Vrati rezultat

PASS ili FAIL status

5

Zaustavi kontejnere

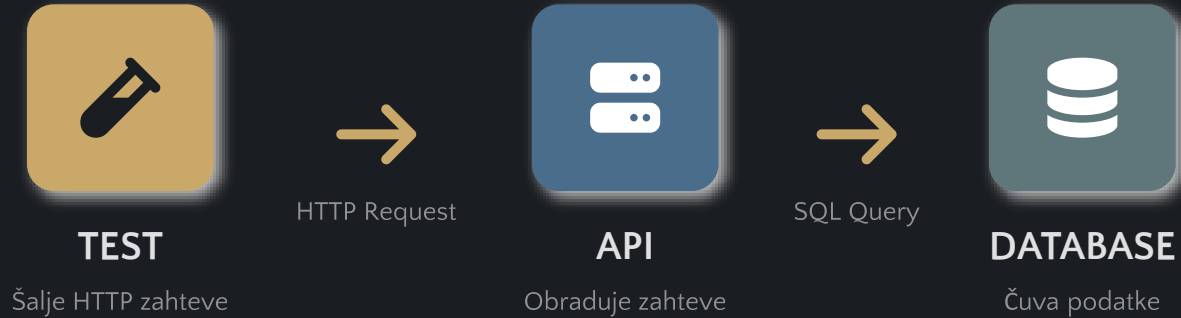
Čisti resurse

🚩 Ključna opcija

```
--abort-on-container-exit
```

Zaustavlja sve kontejnere kada bilo koji kontejner završi. Ovo osigurava da pipeline završi nakon što testovi završe.

Tok Podataka u Sistemu



Docker Osnove

- ✓ Kontejnerizacija
- ✓ Dockerfile
- ✓ Image build



Docker Compose

- ✓ Više servisa
- ✓ Networking
- ✓ Orkestracija



Konfiguracija

- ✓ Env varijable
- ✓ Volumes
- ✓ Port mapping



CI Pipeline

- ✓ GitHub Actions
- ✓ Automatizacija
- ✓ Workflow



Test Automation

- ✓ Jest framework
- ✓ API testovi
- ✓ CI integracija



QA Prakse

- ✓ End-to-end test
- ✓ Izolacija
- ✓ Reproducibilnost

BONUS

Moguća Proširenja



Dodatni testovi

GET test

Proveri da li lista korisnika radi

DELETE test

Proveri brisanje korisnika

Validacija

Proveri error handling



Više tabela

-  orders (porudžbine)
-  products (proizvodi)
-  categories (kategorije)



Health Check (BONUS)

```
app: build: . ports: - "3000:3000" healthcheck: test: ["CMD", "curl", "-f",  
"http://localhost:3000/users"] interval: 5s timeout: 3s retries: 5
```

Health check osigurava da test počne tek kada je API spreman.



Druge ideje

- Autentikacija (JWT tokeni)
- Test coverage report
- Parallel test execution
- Test reports u CI

QA Diskusija - Mogući Problemi



Pitanje

Gde može doći do problema u ovom setup-u?

⚠ Mogući problemi

1

Baza nije spremna

PostgreSQL treba vremena za inicijalizaciju

2

Test počne prerano

depends_on ne čeka da servis bude spreman

3

Env varijable nisu postavljene

API ne može da se poveže na bazu

4

Port konflikt

Port 3000 je već zauzet na mašini

✂ Rešenja



Health checks

Definiši health check za svaki servis



Retry mehanizam

Test retry-uje dok API ne bude dostupan



Validacija .env

Proveri da li su sve varijable postavljene



Dinamički portovi

Koristi random portove u testovima



Ključna lekcija: U QA-u, testovi moraju biti robusni i handlovati sve moguće scenarije.

Rezime Kursa

Student sada zna:



Docker Osnove

Kontejnerizacija, Dockerfile, image build



docker-compose

Orkestracija više servisa, networking



Environment Varijable

Konfiguracija, sigurnost, fleksibilnost



Volumes

Perzistentno skladište podataka



CI Pipeline

GitHub Actions, automatizacija



Testovi u CI

Automatsko pokretanje testova



Praktično Znanje

Student može samostalno postaviti kompletnu QA infrastrukturu sa Docker-om i CI pipeline-om



Realni Scenario

Projekat je baziran na realnim QA zahtevima i industrijskim praksama



Dalji Razvoj

Student može proširiti projekat sa dodatnim testovima i funkcionalnostima

Vaš Zadatak

Student treba da:

1

Pokrene projekat lokalno

docker-compose up --build

2

Pokrene pipeline

Push na GitHub i provera Actions

3

Doda još jedan test

GET, DELETE ili validacija

4

Napravi pull request

Proveri da li pipeline prođe



Rok: 60-120 minuta



Cilj: Praktično iskustvo